

AR SURVIVAL SHOOTER

TECHNICAL DOCUMENT // MUTABAZI ISHIMWE BRUNO

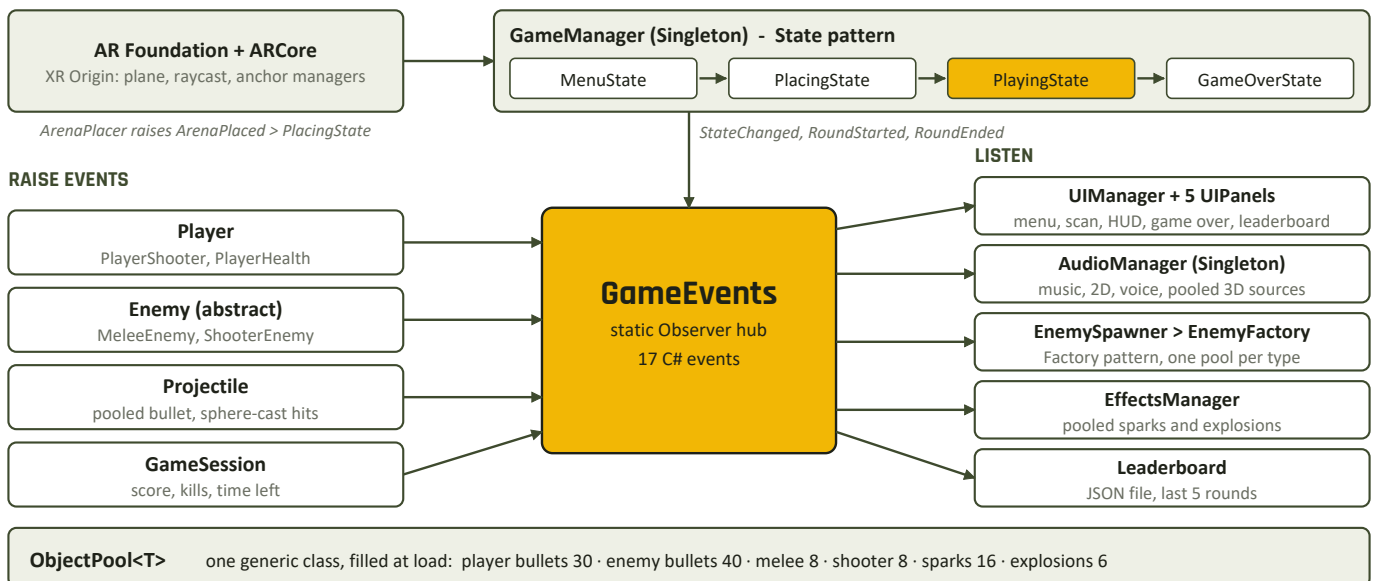
Unity 6000.4.7f1 (URP) · AR Foundation 6.4.3 + Google ARCore XR Plugin 6.4.3 · Input System, TextMeshPro · Target: Android (ARCore phone), IL2CPP, ARM64

Repository: github.com/mutabazi-bruno/Survival-Shooter-Custom-AR-Plane-Tracking-Combat-Experience

1 WHAT THE GAME IS

AR Survival Shooter is a first-person shooter for Android. The player scans the floor, taps once to drop a small war-zone arena onto it, and then has to survive mechs that walk across the room toward them until the timer runs out. The phone is the gun: you aim by moving and turning the phone, and you hold a finger on the screen to fire. There are two enemy types (melee and shooter), two difficulties, a HUD, an end screen and a local leaderboard of the last 5 rounds. The arena is my own layout built from separate free props (wrecked cars, barrels, crates, barricades) on a concrete slab.

2 ARCHITECTURE



The scripts are split by job into folders: AR, Core, Player, Enemies, Combat, Pooling, Audio, Effects, UI, Data and Utils. Two things hold the game together. **GameManager** runs a small state machine with four states (Menu, Placing, Playing, GameOver). Each state is its own class with Enter, Tick and Exit and decides by itself when to move on. **GameEvents** is a static class of C# events (17 in total). Gameplay code only raises events such as *EnemyFired(position)* or *PlayerDied*, and the systems that care (UI, audio, spawner, effects, leaderboard) subscribe to them. A mech does not know that sound or a HUD exist, so each script stays small, and I could add things like hit sparks or the FPS counter later without touching gameplay code.

One round, step by step: Start Mission → PlacingState turns tap-to-place on → the first tap on a horizontal plane places and anchors the arena and stops plane detection → PlayingState creates a new GameSession (score, kills, time) and raises RoundStarted, which resets the player's health, the HUD and starts the spawner → when time runs out or health hits 0, GameManager builds a SessionResult and switches to GameOverState → RoundEnded sends every enemy, bullet and effect back to its pool, the leaderboard saves the round and the end screen shows the result.

3 OBJECT-ORIENTED DESIGN

Principle	Where it is used in my code
Encapsulation	Fields are private and shown in the Inspector with [SerializeField]. Other scripts only read them through properties. Enemy health is private and can only go down through TakeDamage(). GameSession.Score has a private setter, so only AddKill() can change it. DifficultySettings only exposes read-only properties.
Abstraction	Interfaces IDamageable (IsAlive, TakeDamage) and IPoolable (OnTakenFromPool, OnReturnedToPool). A bullet damages "anything damageable" without knowing if it hit the player or a mech, and the pool resets any object without knowing its type. GameState, Enemy and UIPanel are abstract classes.
Inheritance	Enemy → MeleeEnemy, ShooterEnemy (the base holds health, movement, hit flash, death and score). GameState → MenuState, PlacingState, PlayingState, GameOverState. UIPanel → MainMenu, Scan, Hud, GameOver and Leaderboard panels (the base does the fade in and out).
Polymorphism	Enemy.Update() calls the abstract Think(). The melee mech runs in and slashes, while the shooter stops at 1.6 m and fires. GameManager calls currentState.Tick() without knowing which state it is. UIManager calls Show() on any UIPanel and each one overrides OnShow().

4 DESIGN PATTERNS

Pattern	Where	Why I used it
Singleton	GameManager, AudioManager	There is one game flow and one sound system. UI buttons reach them through Instance, and a duplicate destroys itself in Awake.
State	GameState + 4 states	Each phase has its own rules. No big switch statement in Update, and adding a state (for example Pause) means adding one class.
Observer	GameEvents	Keeps gameplay separate from UI, audio, effects and saving. Systems subscribe in OnEnable and unsubscribe in OnDisable.
Factory	EnemyFactory.Create()	The spawner only asks for "a shooter at this point". The factory takes it from the right pool and sets up player, difficulty and return-to-pool.
Object Pool	ObjectPool<T>	No Instantiate or Destroy during a round, so no garbage collection spikes on the phone (section 5).
Data asset	DifficultySettings (ScriptableObject)	Balancing lives in two assets, not in code. Easy: 90 s, 150 HP, spawn every 3.5 s, max 5 alive, 25% shooters, speed ×0.85, damage ×0.75. Hard: 120 s, 100 HP, every 2 s, max 8, 45% shooters, speed ×1.2, damage ×1.25.

5 OBJECT POOLING

All pooling goes through one generic class, `ObjectPool<T>` (Pooling/ObjectPool.cs). The constructor instantiates every object once when the scene loads, disables it and puts it in a queue. **Get()** takes one from the queue, adds it to an in-use list, activates it and calls `OnTakenFromPool()` if the object is `IPoolable`. **Release()** does the opposite and calls `OnReturnedToPool()`. If the pool is empty, `Get()` recycles the oldest object in use instead of creating a new one, so the size is a hard limit. Releasing the same object twice (a bullet that hits something on the same frame its lifetime ends) is ignored. **ReleaseAll()** clears the arena when a round ends. The only Instantiate during play is the arena itself, once, when it is placed.

- **Bullet reset:** lifetime is refilled when taken; damage, direction and the trail are cleared when returned, so a reused bullet never draws a line from its old position.
- **Enemy reset:** health, collider, timers, animation and scale (it grows in from zero). `MeleeEnemy` and `ShooterEnemy` override the reset to also clear their attack timers, then call base.

Pool	Player bullets	Enemy bullets	Melee mechs	Shooter mechs	Sparks	Explosions
Size	30	40	8	8	16	6
Owner	ProjectilePool	ProjectilePool	EnemyFactory	EnemyFactory	EffectsManager	EffectsManager

6 SOUND SYSTEM

AudioManager is the only script that plays sound, and gameplay code never calls it. It subscribes to `GameEvents`. In `Awake` it creates every `AudioSource` once: 1 looping music source that fades between menu and battle music, 1 2D source for the player's own sounds and UI clicks, 1 announcer voice source, a round-robin pool of 10 3D sources for enemy sounds and 4 more just for footsteps (so steps never cut off gunfire). Enemy sounds are placed at the enemy's position (spatial blend 1, linear roll-off from 0.4 to 6 m), so in AR you can hear which side a mech is coming from. Each effect is a **Sound** object holding a few clips, and every play picks a random clip and a slightly different pitch so rapid fire does not repeat.

Required sound	Event it listens to	How it plays	Files
Player shoot	PlayerFired	2D	PlayerShoot_1–3
Player death	PlayerDied	2D	PlayerDeath
Enemy spawn	EnemySpawned	3D, at the spawn point	EnemySpawn
Enemy shoot	EnemyFired	3D, at the mech's gun hand	EnemyShoot_1–2
Melee attack damage	EnemyMeleeHit	3D, at the mech	MeleeHit_1–2

Extra sounds: enemy hit and death, mech footsteps, button click and hover, and an announcer ("Prepare yourself", "Fight", a 5–1 countdown, "You win" / "You lose"). **Sources:** effects and announcer from Kenney's free audio packs (Sci-fi Sounds, Impact Sounds, UI Audio, RPG Audio, Voiceover Pack Fighter, CC0); menu and battle music from Pixabay Music (Pixabay Content License).

7 AR IMPLEMENTATION

- **Plane detection:** `ARPlaneManager` with horizontal detection. Its plane prefab is my *CustomPlane*, with a texture I made showing "Mutabazi Ishimwe Bruno" in stencil. The manager only creates it when `ARCore` finds a plane, and `PlaneTrackerVisual` hides anything that is not facing up, then fades and pulses the texture through a `MaterialPropertyBlock` (no material copies).
- **Tap to place:** `ArenaPlacer` raycasts the tap against `PlaneWithinPolygon`, accepts only `HorizontalUp` planes, turns the arena so enemies come from the far side, attaches an `ARAnchor` so it stays put while tracking improves, then disables the plane manager and hides the planes. Further taps are ignored.
- **Player:** `PlayerHealth`, `PlayerShooter` and the gun sit on the AR camera, because in FPS the phone is the player. Shots are aimed by a ray from the crosshair, so the bullet goes where you look even though it leaves from the gun barrel.

8 PERFORMANCE AND TESTING ON THE PHONE

Tested on a Samsung Android phone with an in-game FPS counter (current, average and lowest FPS per round). ARCore delivers camera frames at 30 FPS and the AR Session's *Match Frame Rate* keeps the game in step with it, so 30 FPS is the ceiling. During a Hard round the counter showed about **27 FPS while playing** and a **23 FPS average** over the whole round. What I did for performance:

- Pooling for every bullet, enemy and effect; MaterialPropertyBlock for hit flashes and plane fading; events instead of Find() or polling; the timer event only fires once per second.
- URP mobile asset: render scale 0.8, MSAA off, HDR off, post-processing off, one shadow cascade with a 6 m shadow distance (it was 50 m, but the arena is only a few metres).
- Problems found on the device and fixed: a black camera feed (Vulkan, fixed by building with OpenGL ES 3 only), a gun that covered half the screen (WeaponView now sizes the gun from the camera's real field of view), and the camera showing through behind the status bar on menus (FillScreen).

9 PROJECT STRUCTURE

Folder (Assets/Scripts)	Scripts	Job
AR	ArenaPlacer, PlaneTrackerVisual	Tap to place with anchor, custom plane look
Core (+ States)	GameManager, GameEvents, GameSession, SessionResult, DifficultySettings, GameState + 4 states	Game flow, events, round data, difficulty
Player	PlayerHealth, PlayerShooter, WeaponView, DamageFlash	Health, shooting, gun on screen, damage feedback
Enemies	Enemy, MeleeEnemy, ShooterEnemy, EnemyFactory, EnemySpawner, ArenaLayout	Enemy behaviour, creation and spawning
Combat / Pooling	Projectile, IDamageable / ObjectPool<T>, IPoolable, ProjectilePool	Bullets, damage contract, pooling
Audio / Effects	AudioManager, Sound / EffectsManager, PooledEffect	All sound, pooled particle effects
UI / Data / Utils	UIManager, UIPanel + 5 panels, SafeArea, FillScreen / Leaderboard / ScreenInput, FpsCounter	Screens, saved scores, input and FPS helpers

10 WHAT I DOWNLOADED AND WHAT I MADE

Downloaded (free, credited in the README)	Made for this project
Mech models and animations: Quaternius Animated Mech Pack (CC0). George is the melee mech, Mike is the shooter. Props: Kenney Car Kit and Survival Kit, Blaster Kit for the gun (CC0), all recoloured to one military palette. Sound: Kenney audio packs (CC0). Music: Pixabay. Fonts: Black Ops One and Rajdhani (Google Fonts, OFL).	The arena: I placed every wreck, barrel, crate and barricade myself on a concrete slab, with smoke, colliders and spawn points. No pre-built environment was used. Plane texture with my name, the UI style and sprites, menu art (rendered in Unity from the mechs), app icon, floor texture, particle effects, animator controllers and all of the code.